

# Variational Autoencoders and Gaussian Mixture

## Variational Autoencoders: A Theoretical Exposition

June 15, 2025

### Abstract

This document provides a detailed theoretical exposition of Variational Autoencoders (VAEs) and their extension, Gaussian Mixture Variational Autoencoders (GMVAEs). It begins by laying the foundational principles of VAEs, contrasting them with standard autoencoders, and detailing their probabilistic framework, architecture, and training process centered around the Evidence Lower Bound (ELBO). The crucial role of the reparameterization trick is explained. The document then transitions to GMVAEs, motivating their development as a solution to the limitations of the unimodal prior in standard VAEs, particularly for modeling multi-modal data and performing unsupervised clustering. The architectural and mathematical modifications required for GMVAEs, including the more complex ELBO and the handling of categorical latent variables, are thoroughly discussed.

# Contents

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| Contents                                                                          | 2         |
| <b>I Variational Autoencoders (VAEs) - Foundations and Principles</b>             | <b>4</b>  |
| <b>1 Introduction to Variational Autoencoders</b>                                 | <b>4</b>  |
| 1.1 Distinction from Standard Autoencoders . . . . .                              | 4         |
| <b>2 VAE Architecture and Components</b>                                          | <b>5</b>  |
| 2.1 The Encoder Network (Inference Network $q_\phi(z   x)$ ) . . . . .            | 5         |
| 2.2 The Latent Space ( $z$ ) . . . . .                                            | 6         |
| 2.3 The Decoder Network (Generative Network $p_\theta(x   z)$ ) . . . . .         | 6         |
| 2.4 The "Sampling" Layer / Reparameterization Trick . . . . .                     | 7         |
| <b>3 Probabilistic Framework of VAEs</b>                                          | <b>7</b>  |
| 3.1 The Prior Distribution $p(z)$ . . . . .                                       | 7         |
| 3.2 The Likelihood $p_\theta(x   z)$ (Decoder/Generative Distribution) . . . . .  | 7         |
| 3.3 The True Posterior $p_\theta(z   x)$ . . . . .                                | 8         |
| 3.4 The Approximate Posterior $q_\phi(z   x)$ (Encoder/Inference Model) . . . . . | 8         |
| <b>4 The Evidence Lower Bound (ELBO): The VAE Objective Function</b>              | <b>8</b>  |
| 4.1 Derivation of the ELBO . . . . .                                              | 9         |
| 4.2 Alternative ELBO Formulation and its Components . . . . .                     | 9         |
| 4.3 The Trade-off . . . . .                                                       | 9         |
| <b>5 Training VAEs: The Reparameterization Trick</b>                              | <b>10</b> |
| 5.1 Necessity for Backpropagation through Stochastic Sampling . . . . .           | 10        |
| 5.2 The Reparameterization Trick Explained . . . . .                              | 10        |
| 5.3 Implementation for a Gaussian Latent Space . . . . .                          | 10        |
| <b>6 Practical Considerations for VAEs (e.g., using MNIST)</b>                    | <b>11</b> |
| 6.1 Training Process . . . . .                                                    | 11        |
| 6.2 Interpreting Test Set Metrics . . . . .                                       | 11        |
| 6.3 Generating New Samples from the Prior $p(z)$ . . . . .                        | 12        |
| <b>II Gaussian Mixture Variational Autoencoders (GMVAEs)</b>                      | <b>12</b> |
| <b>7 Introduction to Gaussian Mixture VAEs</b>                                    | <b>12</b> |
| 7.1 Motivation: Limitations of Unimodal Gaussian Prior in VAEs . . . . .          | 12        |
| 7.2 Modeling Multi-modal Data and Enabling Unsupervised Clustering . . . . .      | 12        |
| <b>8 GMVAE Architecture and Probabilistic Framework</b>                           | <b>13</b> |
| 8.1 The Gaussian Mixture Model (GMM) as the Prior $p(z)$ . . . . .                | 13        |
| 8.2 Introducing a Categorical Latent Variable $y$ . . . . .                       | 13        |
| 8.3 Inference Network Structure for GMVAEs . . . . .                              | 13        |
| 8.4 Decoder $p_\theta(x   z)$ . . . . .                                           | 13        |

|           |                                                                                  |           |
|-----------|----------------------------------------------------------------------------------|-----------|
| <b>9</b>  | <b>The ELBO for GMVAEs: Addressing the KL Divergence</b>                         | <b>14</b> |
| 9.1       | Modified ELBO Formulation . . . . .                                              | 14        |
| 9.2       | Approximation of $D_{\text{KL}}(q_{\phi}(z   x)    p_{\text{GMM}}(z))$ . . . . . | 14        |
| 9.3       | Role of Numerical Stability Functions . . . . .                                  | 14        |
| <b>10</b> | <b>Training GMVAEs</b>                                                           | <b>14</b> |
| 10.1      | Reparameterization for Categorical Variables (Gumbel-Softmax) . . . . .          | 15        |
| 10.2      | Overall Training Procedure . . . . .                                             | 15        |
| <b>11</b> | <b>Applications and Interpretation of GMVAEs</b>                                 | <b>15</b> |
| 11.1      | Unsupervised Clustering Capabilities . . . . .                                   | 15        |
| 11.2      | Interpreting Learned Mixture Components . . . . .                                | 15        |
| 11.3      | Comparison with Standard VAEs . . . . .                                          | 16        |
| <b>12</b> | <b>Conclusion</b>                                                                | <b>16</b> |
|           | <b>References</b>                                                                | <b>16</b> |

## Part I

# Variational Autoencoders (VAEs) - Foundations and Principles

## 1 Introduction to Variational Autoencoders

Variational Autoencoders (VAEs) represent a significant class of generative models within machine learning, distinguished by their capacity to learn a probabilistic representation of input data and subsequently generate novel data instances that exhibit similarity to the training dataset [1]. The fundamental objective of a VAE is to model the underlying probability distribution  $P(x)$  of a high-dimensional dataset  $x$  [3]. This probabilistic approach not only facilitates the generation of new data but also, in principle, allows for the evaluation of the likelihood of new data points, although VAEs are generally more adept at generation than precise likelihood estimation [3]. The core premise is that the observed data  $x$  are samples from some true underlying distribution  $p(x)$ , and the VAE endeavors to maximize the probability of each data point  $X$  in the training set under the complete generative process defined by the model, which involves latent variables  $z$  and model parameters  $\theta$  [4].

### 1.1 Distinction from Standard Autoencoders

To appreciate the unique characteristics of VAEs, it is instructive to compare them with standard autoencoders (AEs). AEs are neural network architectures primarily employed for tasks such as dimensionality reduction, data compression, and denoising [2]. They function by mapping high-dimensional input data to a lower-dimensional latent space and then decoding this latent representation back to the original data space, aiming to reconstruct the input as accurately as possible [4]. While AEs can effectively learn salient features of the data, their latent space is typically deterministic and often lacks the regular structure required for robust generative tasks [2]. The encoder in a standard AE produces a single, fixed latent vector for each input.

In stark contrast, VAEs introduce a probabilistic interpretation to the latent space. Instead of mapping an input to a fixed point, the VAE encoder outputs parameters—typically the mean  $\mu$  and the standard deviation  $\sigma$  (or log-variance)—of a probability distribution (commonly Gaussian) in the latent space [1]. This probabilistic encoding is the cornerstone of their generative capabilities. The decoder in a VAE is not merely an auxiliary component for training the encoder; it is an integral part of the generative process, retained and utilized to synthesize new data points from samples drawn from the latent distribution [2]. Standard AEs, on the other hand, often discard the decoder after the encoder has been trained for feature extraction.

The transition from a deterministic latent point in AEs to a probabilistic latent distribution in VAEs constitutes a fundamental shift that underpins the generative capacity of VAEs. Standard AEs might learn an effective compressed representation, but the space between these encoded points can be devoid of meaning; decoding from such intermediate points does not guarantee sensible outputs, as the decoder is only trained on specific, discrete codes [4]. VAEs, however, impose structure and continuity on the latent space, primarily through the Kullback-Leibler (KL) divergence term in their loss function, by compelling the encoder to output parameters of a distribution [2]. Sampling from this learned distribution—even points not directly corresponding to the mean of any specific training input—and subsequently decoding these samples allows for

the generation of novel yet coherent data instances. This is feasible because the decoder learns to map entire regions of the latent space back to the data space, rather than isolated points. This probabilistic nature is thus the direct cause of the VAE’s generative prowess, enabling it to learn the underlying manifold of the data in a probabilistic manner.

Furthermore, VAEs can be understood as an elegant integration of probabilistic graphical models with the representational power of deep neural networks [5]. The encoder network effectively performs amortized variational inference, learning a mapping  $q_\phi(z | x)$  that approximates the true, often intractable, posterior distribution  $p(z | x)$  [4]. Concurrently, the decoder network learns the parameters of the generative distribution  $p_\theta(x | z)$  [4]. This framework—comprising latent variables, prior distributions, likelihood functions, and posterior inference—is characteristic of Bayesian graphical models. Neural networks serve as potent function approximators for both the encoder and decoder components [4]. This amalgamation allows VAEs to effectively model complex, high-dimensional data, such as images, where traditional probabilistic graphical models might encounter difficulties in defining or learning the intricate conditional probability distributions. This synergy between probabilistic modeling and deep learning is an increasingly prominent theme in the development of sophisticated and flexible machine learning models.

Table 1: Comparison of Standard Autoencoder vs. Variational Autoencoder

| Feature                            | Standard Autoencoder                                       | Variational Autoencoder                                                       |
|------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Latent Space Representation</b> | Deterministic vector                                       | Parameters of a probability distribution (e.g., $\mu, \sigma$ for a Gaussian) |
| <b>Encoder Output</b>              | Single latent vector $z$                                   | Parameters (e.g., $\mu(x), \sigma(x)$ ) defining $q_\phi(z   x)$              |
| <b>Decoder Purpose</b>             | Reconstruct input; often discarded after training          | Reconstruct input; integral part of the generative process                    |
| <b>Primary Goal</b>                | Dimensionality reduction, feature learning, reconstruction | Generative modeling, learning a probabilistic latent space, reconstruction    |
| <b>Generative Capability</b>       | Limited; latent space often not smooth or continuous       | Designed for generation; learns a smooth, continuous latent space             |
| <b>Regularization</b>              | Typically via bottleneck size, weight decay, sparsity      | KL divergence between $q_\phi(z   x)$ and a prior $p(z)$                      |

## 2 VAE Architecture and Components

A Variational Autoencoder is composed of several key architectural and functional components that work in concert to achieve its generative capabilities.

### 2.1 The Encoder Network (Inference Network $q_\phi(z | x)$ )

The encoder, also referred to as the inference or recognition network, is responsible for processing the input data  $x$  and learning its salient features [1]. Its primary function is to output the

parameters of a probability distribution in the latent space. For a Gaussian latent space, these parameters are typically the mean vector  $\mu$  and the log-variance vector  $\log(\sigma^2)$  (or equivalently, the standard deviation  $\sigma$ ) for each dimension of the latent space [1]. The choice of log-variance is a common practical consideration, as  $\log(\sigma^2)$  can assume any real value, whereas variance itself must be positive; the standard deviation  $\sigma$  is then computed as  $\exp(0.5 \times \log(\sigma^2))$ .

Architecturally, the encoder is usually implemented as a neural network. For simpler, vector-based data, fully connected layers are often employed, while for more complex data like images, convolutional neural networks (CNNs) are preferred due to their ability to capture spatial hierarchies [2]. A crucial aspect of the VAE encoder is that it performs "variational inference" in an amortized fashion. This means that instead of optimizing a separate variational distribution  $q(z | x)$  for each individual data point  $x$ , the encoder learns a parameterized function that maps any input  $x$  directly to the parameters of its corresponding  $q_\phi(z | x)$  [5]. This makes the inference process highly efficient for new data points.

## 2.2 The Latent Space ( $z$ )

The latent space  $z$  is a lower-dimensional, continuous space where the compressed, probabilistic representation of the input data resides [1]. Unlike standard autoencoders that map inputs to a fixed point in this space, VAEs characterize each input by a distribution over the latent space, or more precisely, the parameters of  $q_\phi(z | x)$  which are guided by a prior  $p(z)$  [2]. For effective generative modeling, the latent space must possess certain desirable properties [2]:

- **Continuity:** Points that are proximate in the latent space should, upon decoding, yield outputs that are semantically similar and vary smoothly.
- **Completeness (or Well-structuredness):** Sampling points from regions of the latent space that were adequately "explored" or populated during training should result in meaningful and coherent decoded outputs. The KL divergence component of the VAE's loss function plays a vital role in enforcing this structure.

The dimensionality of the latent space is a critical hyperparameter. An excessively small latent space may fail to capture the full variability of the data, leading to underfitting. Conversely, an overly large latent space might result in overfitting or diminish the effectiveness of the regularization, potentially leading to a less structured space. The general concept of a latent space involves a compressed representation that preserves only the essential features informing the underlying structure of the input data [7].

## 2.3 The Decoder Network (Generative Network $p_\theta(x | z)$ )

The decoder, also known as the generative network, takes a sample  $z$  drawn from the latent space distribution and endeavors to reconstruct the original input data  $x$  [1]. Architecturally, the decoder is typically a neural network that mirrors the encoder's structure but in reverse, with layers progressively increasing the dimensionality of the representation to match that of the original input space [2].

The output of the decoder defines the parameters of the likelihood distribution  $p_\theta(x | z)$ . The specific form of this distribution is chosen based on the characteristics of the data  $x$ . For instance, if the input data consists of binary images (e.g., binarized MNIST digits), the decoder might output the parameters (pixel-wise probabilities) of a Bernoulli distribution [8]. For real-valued data, such as images with normalized pixel intensities, the decoder might output the

mean of a Gaussian distribution, often assuming a fixed or shared variance [3]. The decoder effectively learns to map arbitrary latent coordinates back to distributions over the original data space [4].

## 2.4 The "Sampling" Layer / Reparameterization Trick

Positioned conceptually between the encoder and the decoder, the sampling layer acts as the bottleneck. It receives the mean and log-variance parameters from the encoder and is responsible for generating latent vectors  $z$  [1]. This sampling is performed using the reparameterization trick:  $z = \mu + \sigma \cdot \epsilon$ , where  $\epsilon$  is a noise vector sampled from a standard normal distribution (e.g.,  $\mathcal{N}(0, I)$ ) [1]. This technique is indispensable for enabling backpropagation through the stochastic sampling process and will be elaborated upon in Section 5. It is this layer that introduces the controlled randomness necessary for the VAE to generate diverse outputs from similar inputs [1].

The encoder's output parameters  $(\mu, \sigma)$  and the subsequent application of the reparameterization trick are not merely architectural details; they represent the core mechanism that connects deterministic neural network components with the principles of probabilistic latent variable modeling. The reparameterization trick,  $z = \mu + \sigma \cdot \epsilon$ , ingeniously disentangles the deterministic part of this process (which depends on the learnable parameters  $\mu$  and  $\sigma$ ) from the stochastic part ( $\epsilon$ , which is drawn from a fixed distribution and does not depend on network parameters) [1]. This separation allows gradients to flow back to  $\mu$  and  $\sigma$ , and consequently to the weights of the encoder network, thereby enabling the end-to-end training of a probabilistic model using standard deep learning optimization techniques.

## 3 Probabilistic Framework of VAEs

The operation and learning process of VAEs are deeply rooted in a probabilistic framework, involving several key distributions that define the model's generative process and the inference mechanism.

### 3.1 The Prior Distribution $p(z)$

The prior distribution  $p(z)$  represents a pre-specified belief about the distribution of the latent variables  $z$  before any data is observed [12]. It is a fundamental component of the Bayesian approach underlying VAEs. In practice,  $p(z)$  is typically chosen to be a simple, tractable distribution, with the most common choice being a standard multivariate Gaussian distribution:  $p(z) = \mathcal{N}(z | 0, I)$ , where  $I$  is the identity covariance matrix [2]. This selection offers significant advantages, primarily because it simplifies the calculation of the KL divergence term in the VAE's objective function, often allowing for an analytical, closed-form solution [14].

### 3.2 The Likelihood $p_\theta(x | z)$ (Decoder/Generative Distribution)

The likelihood function  $p_\theta(x | z)$  quantifies the probability of observing the data  $x$  given a particular latent variable  $z$ . This distribution is parameterized by the weights  $\theta$  of the decoder network and essentially defines the generative part of the VAE [4]. The specific mathematical form of the likelihood function is chosen based on the nature of the input data  $x$ :

- For binary data, a Bernoulli distribution is commonly used. In this case,  $p_\theta(x_i | z) = \text{Bernoulli}(\hat{x}_i(z))$ , where  $\hat{x}_i(z)$  is the  $i$ -th output of the decoder [6].

- For real-valued data, a Gaussian distribution is often employed:  $p_\theta(x | z) = \mathcal{N}(x | \mu_\theta(z), \sigma^2 I)$ . Here,  $\mu_\theta(z)$  is the output of the decoder network, and the covariance  $\sigma^2 I$  signifies that pixels are often assumed to be conditionally independent given  $z$  [3].

The reconstruction loss component of the VAE's objective function is derived from the negative log-likelihood,  $-\log p_\theta(x | z)$ .

### 3.3 The True Posterior $p_\theta(z | x)$

The true posterior distribution  $p_\theta(z | x)$  represents the probability of the latent variables  $z$  given the observed data  $x$  and the parameters  $\theta$  of the generative model. It can be expressed using Bayes' rule:

$$p_\theta(z | x) = \frac{p_\theta(x | z)p(z)}{p_\theta(x)} \quad (1)$$

The denominator,  $p_\theta(x) = \int p_\theta(x | z)p(z)dz$ , is the marginal likelihood of the data, often referred to as the "evidence." For complex models like VAEs, computing this integral is generally intractable because it requires integrating over all possible values of the high-dimensional latent variable  $z$  [4].

### 3.4 The Approximate Posterior $q_\phi(z | x)$ (Encoder/Inference Model)

Due to the intractability of the true posterior  $p_\theta(z | x)$ , VAEs employ a core technique from variational inference: they introduce an approximate posterior distribution  $q_\phi(z | x)$ . This distribution is parameterized by the weights  $\phi$  of the encoder network [4]. The key idea is to choose a family of distributions for  $q_\phi(z | x)$  that is tractable and flexible enough to approximate the true posterior reasonably well. Typically,  $q_\phi(z | x)$  is chosen to be a Gaussian distribution whose mean and covariance (often diagonal) are outputs of the encoder network:  $q_\phi(z | x) = \mathcal{N}(z | \mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$  [8]. The goal of the VAE training process is then to make this approximate posterior  $q_\phi(z | x)$  as close as possible to the true posterior  $p_\theta(z | x)$ .

## 4 The Evidence Lower Bound (ELBO): The VAE Objective Function

The training of a VAE revolves around optimizing the Evidence Lower Bound (ELBO), which serves as a tractable surrogate for the intractable marginal log-likelihood of the data.



## 4.1 Derivation of the ELBO

The primary objective is to maximize the marginal log-likelihood  $\log p_\theta(x)$ . By introducing the approximate posterior  $q_\phi(z | x)$ , we can write:

$$\begin{aligned}\log p_\theta(x) &= \int q_\phi(z | x) \log p_\theta(x) dz \\ &= \int q_\phi(z | x) \log \left( \frac{p_\theta(x, z)}{p_\theta(z | x)} \right) dz \\ &= \int q_\phi(z | x) \log \left( \frac{p_\theta(x, z)}{q_\phi(z | x)} \frac{q_\phi(z | x)}{p_\theta(z | x)} \right) dz \\ &= \int q_\phi(z | x) \log \left( \frac{p_\theta(x, z)}{q_\phi(z | x)} \right) dz + \int q_\phi(z | x) \log \left( \frac{q_\phi(z | x)}{p_\theta(z | x)} \right) dz \\ &= \mathcal{L}(\theta, \phi; x) + D_{\text{KL}}(q_\phi(z | x) || p_\theta(z | x))\end{aligned}$$

The first term is the ELBO, denoted  $\mathcal{L}(\theta, \phi; x)$ , and the second term is the Kullback-Leibler (KL) divergence between the approximate and true posteriors. Since KL divergence is always non-negative, we have  $\log p_\theta(x) \geq \mathcal{L}(\theta, \phi; x)$ . Maximizing the ELBO thus pushes up a lower bound on the log-likelihood of the data [12, 13].

## 4.2 Alternative ELBO Formulation and its Components

The ELBO can be rewritten in a more intuitive form by expanding  $p_\theta(x, z) = p_\theta(x | z)p(z)$ :

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x | z) + \log p(z) - \log q_\phi(z | x)] \quad (2)$$

$$= \underbrace{\mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x | z)]}_{\text{Reconstruction Term}} - \underbrace{D_{\text{KL}}(q_\phi(z | x) || p(z))}_{\text{Regularization Term}} \quad (3)$$

This formulation clearly delineates the two primary components of the VAE objective:

1. **Reconstruction Loss (Expected Log-Likelihood):** The first term,  $\mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x | z)]$ , encourages the decoder to accurately reconstruct the input data  $x$ . Maximizing this term is equivalent to minimizing a reconstruction error, such as Binary Cross-Entropy (BCE) for binary data or Mean Squared Error (MSE) for real-valued data [6, 1].
2. **KL Divergence (Regularization Term):** The second term,  $-D_{\text{KL}}(q_\phi(z | x) || p(z))$ , acts as a regularizer. It forces the learned latent distributions  $q_\phi(z | x)$  to be close to the chosen prior  $p(z)$ . This encourages a well-structured, continuous latent space [2]. For the common case where  $q_\phi(z | x) = \mathcal{N}(z | \mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$  and  $p(z) = \mathcal{N}(0, I)$ , this KL divergence has a convenient analytical form:

$$D_{\text{KL}}(q_\phi(z | x) || p(z)) = -\frac{1}{2} \sum_{j=1}^J (1 + \log(\sigma_{\phi,j}^2(x)) - \mu_{\phi,j}^2(x) - \sigma_{\phi,j}^2(x)) \quad (4)$$

where  $J$  is the dimensionality of the latent space [1].

## 4.3 The Trade-off

The VAE loss function inherently involves a trade-off between these two terms. Heavy weighting on reconstruction can lead to an irregular latent space, while heavy weighting on the KL term can lead to "posterior collapse," where the latent variables become uninformative, resulting in blurry reconstructions [16]. The  $\beta$ -VAE model explicitly introduces a weighting factor  $\beta$  to control this balance [5].

Table 2: Key Components of the ELBO in VAEs

| Term Name                                            | Mathematical Expression                                | Expression | Role/Purpose in VAE                                                                                                                                                       |
|------------------------------------------------------|--------------------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Expected Log-Likelihood (Reconstruction Term)</b> | $\mathbb{E}_{z \sim q_\phi(z x)} [\log p_\theta(x z)]$ |            | Encourages the decoder to accurately reconstruct the input data $x$ from its latent representation $z$ . Maximizes the likelihood of the data given the latent code.      |
| <b>KL Divergence (Regularization Term)</b>           | $-D_{\text{KL}}(q_\phi(z x)    p(z))$                  |            | Acts as a regularizer, forcing the approximate posterior $q_\phi(z x)$ to be close to the prior $p(z)$ . This promotes a smooth, continuous, and structured latent space. |

## 5 Training VAEs: The Reparameterization Trick

A critical innovation that enables the effective training of VAEs using standard gradient-based optimization methods is the reparameterization trick.

### 5.1 Necessity for Backpropagation through Stochastic Sampling

The VAE training objective involves an expectation with respect to  $q_\phi(z|x)$ . To estimate this, samples  $z$  must be drawn from  $q_\phi(z|x)$ . However, the sampling operation is non-differentiable with respect to the parameters  $\phi$  that define the distribution. This breaks the computation graph, preventing the encoder from learning via backpropagation [10].

### 5.2 The Reparameterization Trick Explained

The reparameterization trick reformulates the sampling process by expressing the random variable  $z \sim q_\phi(z|x)$  as a deterministic function of the parameters  $\phi$  and an auxiliary independent noise variable  $\epsilon$ , whose distribution  $p(\epsilon)$  does not depend on  $\phi$ . That is,  $z$  is generated as  $z = g(\phi, \epsilon)$ , where  $\epsilon \sim p(\epsilon)$  [10].

This moves the stochasticity outside the part of the graph that depends on learnable parameters. The gradient of the expectation can now be moved inside the expectation:

$$\nabla_\phi \mathbb{E}_{\epsilon \sim p(\epsilon)} [f(g(\phi, \epsilon))] = \mathbb{E}_{\epsilon \sim p(\epsilon)} [\nabla_\phi f(g(\phi, \epsilon))] \quad (5)$$

This allows for low-variance Monte Carlo estimates of the gradient by sampling  $\epsilon$  and backpropagating through the deterministic functions  $g$  and  $f$ .

### 5.3 Implementation for a Gaussian Latent Space

For a diagonal Gaussian posterior  $q_\phi(z|x) = \mathcal{N}(z | \mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$ , the reparameterization trick is implemented as follows:

1. Sample a noise vector  $\epsilon$  from a standard multivariate Gaussian:  $\epsilon \sim \mathcal{N}(0, I)$ .
2. Compute the latent vector  $z$  using the transformation:  $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ , where  $\odot$  denotes element-wise multiplication [1].

In this formulation,  $\mu_\phi(x)$  and  $\sigma_\phi(x)$  are deterministic outputs of the encoder. Gradients can now flow back through this transformation to the encoder parameters  $\phi$ , enabling end-to-end training.

## 6 Practical Considerations for VAEs (e.g., using MNIST)

Training and utilizing VAEs involve several practical steps, often demonstrated with the MNIST dataset [6].

### 6.1 Training Process

1. **Data Preparation:** Load and normalize the dataset (e.g., MNIST pixel values to  $[0, 1]$ ) [1]. Reshape data as needed for the network architecture (e.g., flatten for dense layers, add channel dimension for CNNs) [6].
2. **Model Definition:**
  - **Encoder:** A neural network (e.g., Dense or Conv layers) that outputs parameters  $\mu$  and  $\log(\sigma^2)$  [1, 9].
  - **Sampling Layer:** Implements the reparameterization trick  $z = \mu + \sigma \odot \epsilon$  [1].
  - **Decoder:** A network, often symmetric to the encoder, that reconstructs the input from  $z$ . The output activation (e.g., sigmoid) must match the data distribution and loss function [1].
3. **Loss Function (Negative ELBO):** The total loss is the sum of the reconstruction loss and the KL divergence.
  - **Reconstruction Loss:** For MNIST, typically Binary Cross-Entropy (BCE) between input  $x$  and reconstruction  $\hat{x}$  [6].
  - **KL Divergence:** Calculated analytically using the formula for two Gaussians [1].
4. **Optimization:** Use an optimizer like Adam to minimize the total loss over several epochs [6].

### 6.2 Interpreting Test Set Metrics

Evaluating the VAE on a held-out test set provides insights:

- **Negative ELBO (Test Set):** The primary metric for overall generalization. Lower is better.
- **Reconstruction Loss (Test Set):** Measures reconstruction quality on unseen data. High values may indicate underfitting or an over-dominant KL term.
- **KL Divergence (Test Set):** Measures the deviation of latent distributions from the prior. A value near zero with high reconstruction loss can indicate posterior collapse [16].

### 6.3 Generating New Samples from the Prior $p(z)$

Once trained, new data is generated by:

1. Sampling latent vectors  $z_{\text{new}}$  directly from the prior distribution, e.g.,  $z_{\text{new}} \sim \mathcal{N}(0, I)$ .
2. Passing these samples through the trained decoder network:  $x_{\text{generated}} = \text{decoder}(z_{\text{new}})$  [1, 27].

The resulting  $x_{\text{generated}}$  are new data samples that should resemble the training data.

## Part II

# Gaussian Mixture Variational Autoencoders (GMVAEs)

## 7 Introduction to Gaussian Mixture VAEs

Gaussian Mixture Variational Autoencoders (GMVAEs) are an extension of the standard VAE framework, designed to address some of the limitations inherent in using a simple unimodal Gaussian prior, particularly when modeling data with complex, multi-modal underlying structures or when aiming for unsupervised clustering.

### 7.1 Motivation: Limitations of Unimodal Gaussian Prior in VAEs

Standard VAEs typically employ a unimodal prior, most commonly  $p(z) = \mathcal{N}(0, I)$  [28]. This choice, while computationally convenient, encourages the learned approximate posteriors  $q_\phi(z | x)$  to be unimodal and concentrated around the origin. Such a prior can be restrictive when the true underlying structure of the data is inherently multi-modal or when a primary goal is to discover distinct clusters [16]. A single Gaussian may lack the expressive capacity to adequately capture such complex latent distributions [16, 29].

### 7.2 Modeling Multi-modal Data and Enabling Unsupervised Clustering

GMVAEs address this limitation by employing a Gaussian Mixture Model (GMM) as the prior distribution for the latent space  $z$  [16]. A GMM prior is defined as a weighted sum of several Gaussian components:  $p(z) = \sum_{k=1}^K \pi_k \mathcal{N}(z | \mu_k, \Sigma_k)$ , where  $K$  is the number of mixture components. Each component can, in principle, represent a different mode or cluster present in the data.

The use of a GMM prior allows the GMVAE to learn a latent space where different clusters of data points are mapped to distinct components of the GMM. This inherently facilitates unsupervised clustering as part of the model's learning process [28]. Consequently, a GMVAE can simultaneously perform dimensionality reduction and clustering within a single, unified framework [29, 32]. The work by Dilokthanakul et al. explicitly focuses on "Deep Unsupervised Clustering with Gaussian Mixture Variational Autoencoders" [30].

## 8 GMVAE Architecture and Probabilistic Framework

The architecture and probabilistic underpinnings of GMVAEs build upon the standard VAE framework but introduce key modifications to accommodate the GMM prior.

### 8.1 The Gaussian Mixture Model (GMM) as the Prior $p(z)$

The defining characteristic of a GMVAE is its prior over the continuous latent variables  $z$ . Instead of a single Gaussian, a GMM is used:

$$p(z) = \sum_{k=1}^K p(y = k) p(z | y = k) \quad (6)$$

Here,  $y$  is an auxiliary categorical latent variable indicating the mixture component.

- $p(y = k) = \pi_k$  are the mixture weights, with  $\pi_k \geq 0$  and  $\sum_{k=1}^K \pi_k = 1$ .
- $p(z | y = k) = \mathcal{N}(z | \mu_k, \Sigma_k)$  is the  $k$ -th Gaussian component.

The GMM parameters  $(\pi_k, \mu_k, \Sigma_k)$  can be fixed or learned during training [16, 34].

### 8.2 Introducing a Categorical Latent Variable $y$

To model this mixture prior, a discrete latent variable  $y \in \{1, \dots, K\}$  is explicitly introduced to represent the choice of the Gaussian component [28]. The generative process is hierarchical:

1. Sample the component index  $y \sim \text{Categorical}(\pi)$ .
2. Sample the continuous latent vector  $z \sim p(z | y) = \mathcal{N}(z | \mu_y, \Sigma_y)$ .
3. Sample the observed data  $x \sim p_\theta(x | z)$ .

### 8.3 Inference Network Structure for GMVAEs

The inference network (encoder),  $q_\phi(y, z | x)$ , must approximate the true posterior  $p(y, z | x)$ . It is often factorized for tractability, e.g., as  $q_\phi(y, z | x) = q_\phi(y | x) q_\phi(z | x, y)$ .

- $q_\phi(y | x)$ : This part of the network takes input  $x$  and predicts the probability distribution over the mixture components, typically using a softmax output layer to produce responsibilities  $q_\phi(y = k | x)$  [25].
- $q_\phi(z | x, y)$  (or  $q_\phi(z | x)$ ): This is the encoder for the continuous latent variable  $z$ . It outputs the parameters of a Gaussian distribution for  $z$ . In simpler implementations, a single encoder  $q_\phi(z | x) = \mathcal{N}(z | \mu_\phi(x), \Sigma_\phi(x))$  is used, and the inference over  $y$  is handled implicitly through the KL divergence term [30].

### 8.4 Decoder $p_\theta(x | z)$

The decoder network  $p_\theta(x | z)$  in a GMVAE functions similarly to that in a standard VAE. It takes a latent sample  $z$  and maps it back to the data space to reconstruct the original input  $x$ .

## 9 The ELBO for GMVAEs: Addressing the KL Divergence

The ELBO for GMVAEs retains the same fundamental structure but requires careful consideration of the KL divergence term due to the GMM prior.

### 9.1 Modified ELBO Formulation

The general form of the ELBO is  $\mathcal{L} = \mathbb{E}_{q_\phi(y, z | x)} [\log p_\theta(x | z)] - D_{\text{KL}}(q_\phi(y, z | x) || p(y, z))$ . Assuming the factorization  $q_\phi(y, z | x) = q_\phi(y | x)q_\phi(z | x, y)$ , the ELBO can be expanded to:

$$\mathcal{L} = \sum_{k=1}^K q_\phi(y = k | x) \left( \mathbb{E}_{q_\phi(z | x, y=k)} [\log p_\theta(x | z)] - D_{\text{KL}}(q_\phi(z | x, y = k) || p(z | y = k)) \right) - D_{\text{KL}}(q_\phi(y | x) || p(y)) \quad (7)$$

A more common simplification, if the encoder  $q_\phi(z | x)$  outputs a single Gaussian, is:

$$\mathcal{L} = \mathbb{E}_{q_\phi(z | x)} [\log p_\theta(x | z)] - D_{\text{KL}}(q_\phi(z | x) || p_{\text{GMM}}(z)) \quad (8)$$

where  $p_{\text{GMM}}(z) = \sum_{k=1}^K \pi_k \mathcal{N}(z | \mu_k, \Sigma_k)$ .

### 9.2 Approximation of $D_{\text{KL}}(q_\phi(z | x) || p_{\text{GMM}}(z))$

Let  $q(z)$  be the Gaussian posterior  $q_\phi(z | x)$  and  $p(z)$  be the GMM prior. The KL divergence is:

$$D_{\text{KL}}(q(z) || p(z)) = \int q(z) \log \frac{q(z)}{p(z)} dz = \mathbb{E}_{z \sim q(z)} [\log q(z) - \log p(z)] \quad (9)$$

The term  $\mathbb{E}_{z \sim q(z)} [\log p(z)] = \mathbb{E}_{z \sim q(z)} \left[ \log \sum_{k=1}^K \pi_k \mathcal{N}(z | \mu_k, \Sigma_k) \right]$  does not have a closed-form solution due to the logarithm of a sum [38].

**Approximation Method:** The most common approach is **Monte Carlo Estimation**. Samples  $z^{(s)}$  are drawn from the posterior  $q(z)$ , and the expectation is approximated by the sample mean:

$$\mathbb{E}_{z \sim q(z)} [\log p(z)] \approx \frac{1}{S} \sum_{s=1}^S \log \left( \sum_{k=1}^K \pi_k \mathcal{N}(z^{(s)} | \mu_k, \Sigma_k) \right) \quad (10)$$

This introduces variance into the gradient estimates but is a general and effective method [38].

### 9.3 Role of Numerical Stability Functions

Implementing the GMVAE ELBO requires careful handling of probabilities to avoid numerical underflow or overflow.

- `log_normal(z, mu, sigma_sq)`: Computes the log-probability density  $\log \mathcal{N}(z | \mu, \Sigma)$  [40].
- `log_sum_exp(x_1, ..., x_N)`: Computes  $\log(\sum_{i=1}^N \exp(x_i))$  in a numerically stable way. This is crucial for calculating  $\log p_{\text{GMM}}(z)$  from the log-densities of its components [43]. The trick is to compute  $m + \log(\sum_{i=1}^N \exp(x_i - m))$ , where  $m = \max(x_i)$ .

## 10 Training GMVAEs

Training GMVAEs involves optimizing the ELBO using stochastic gradient descent, with additional considerations for the mixture prior.

## 10.1 Reparameterization for Categorical Variables (Gumbel-Softmax)

If the model explicitly samples the categorical variable  $y$ , a reparameterization trick for discrete variables is needed to allow gradient flow. The **Gumbel-Softmax trick** (or Concrete distribution) provides a continuous, differentiable approximation to sampling from a categorical distribution [10]. To sample a  $K$ -dimensional vector  $y$  approximating a one-hot sample from a categorical distribution with probabilities  $\pi$ :

$$y_k = \frac{\exp((\log \pi_k + g_k)/\tau)}{\sum_{j=1}^K \exp((\log \pi_j + g_j)/\tau)} \quad \text{for } k = 1, \dots, K \quad (11)$$

where  $g_k$  are i.i.d. samples from a Gumbel(0,1) distribution and  $\tau$  is a temperature parameter that is typically annealed during training [25].

## 10.2 Overall Training Procedure

1. **Model Components:** Define encoder(s) and decoder as neural networks.
2. **ELBO Calculation:** For each mini-batch, compute the GMVAE ELBO. This may involve Monte Carlo estimation for the KL term.
3. **Backpropagation:** Calculate gradients of the negative ELBO with respect to all learnable parameters ( $\phi, \theta$ , and possibly GMM parameters). This uses the standard reparameterization trick for  $z$  and the Gumbel-Softmax trick for  $y$ , if applicable.
4. **Parameter Updates:** Update parameters using an optimizer like Adam.

Training GMVAEs can be more sensitive than standard VAEs due to the variance from Monte Carlo estimation and the need to manage additional hyperparameters like the temperature  $\tau$  and the number of mixture components  $K$  [31, 38].

# 11 Applications and Interpretation of GMVAEs

GMVAEs are primarily applied in unsupervised clustering and in generating data from specific learned categories.

## 11.1 Unsupervised Clustering Capabilities

A principal application of GMVAEs is unsupervised clustering [28]. Once trained, the inference network  $q_\phi(y | x)$  can be used to assign a data point  $x$  to the cluster  $k$  for which the posterior probability  $q_\phi(y = k | x)$  is maximized. This has been successfully applied to clustering molecular simulation data [29] and video game levels [32].

## 11.2 Interpreting Learned Mixture Components

Each Gaussian component in the GMM prior is intended to correspond to a distinct data category. This offers enhanced interpretability:

- By sampling a latent vector  $z$  from a specific learned component  $k$  and passing it through the decoder, one can generate new data samples that belong specifically to that category [32].
- Visualizing the latent space can reveal how different data points form distinct clusters around the means of the GMM components.

### 11.3 Comparison with Standard VAEs

Table 3: Comparison of Standard VAE vs. Gaussian Mixture VAE

| Feature                      | Standard VAE                                         | Gaussian Mixture VAE (GMVAE)                             |
|------------------------------|------------------------------------------------------|----------------------------------------------------------|
| Prior Distribution Type      | Unimodal Gaussian (typically $\mathcal{N}(0, I)$ )   | Gaussian Mixture Model (GMM)                             |
| Latent Space Structure       | Learns a continuous, generally unimodal latent space | Learns a multi-modal latent space with distinct regions  |
| Native Clustering Capability | Not explicitly designed for clustering               | Performs unsupervised clustering inherently              |
| KL Divergence Complexity     | Often analytical and simple (Gaussian vs. Gaussian)  | More complex; requires approximation (e.g., Monte Carlo) |
| Model Expressiveness         | Limited for multi-modal data                         | More expressive; suited for multi-modal data             |
| Typical Use Cases            | General data generation, smooth interpolation        | Unsupervised clustering, category-specific generation    |

This comparison highlights that while standard VAEs excel at learning smooth latent spaces, GMVAEs offer a distinct advantage when the data has a multi-modal structure or when unsupervised clustering is a primary objective [29].

## 12 Conclusion

Variational Autoencoders (VAEs) and their extension, Gaussian Mixture Variational Autoencoders (GMVAEs), represent powerful frameworks within deep generative modeling. VAEs provide a principled approach to learning probabilistic latent representations, balancing reconstruction fidelity with latent space regularization via the ELBO. The reparameterization trick is a key innovation enabling their training.

Standard VAEs, however, can be limited by their unimodal Gaussian prior. GMVAEs address this by employing a GMM prior, allowing them to model multi-modal data distributions and perform unsupervised clustering inherently. This increased expressiveness comes at the cost of greater model complexity, particularly in the calculation of the KL divergence and the potential need to handle categorical latent variables.

The choice between a VAE and a GMVAE depends on the data’s characteristics and the modeling goals. For unimodal data or smooth generation, a VAE may suffice. For multi-modal data or when cluster discovery is desired, a GMVAE offers a more tailored and expressive approach. The principles discussed herein provide the foundational knowledge to implement, evaluate, and apply these sophisticated generative models to a wide range of problems.

## References

- [1] Placeholder reference 1.
- [2] Placeholder reference 2.



- [3] Placeholder reference 3.
- [4] Placeholder reference 4.
- [5] Placeholder reference 5.
- [6] Placeholder reference 6.
- [7] Placeholder reference 7.
- [8] Placeholder reference 8.
- [9] Placeholder reference 9.
- [10] Placeholder reference 10.
- [11] Placeholder reference 11.
- [12] Placeholder reference 12.
- [13] Placeholder reference 13.
- [14] Placeholder reference 14.
- [15] Placeholder reference 15.
- [16] Placeholder reference 16.
- [17] Placeholder reference 17.
- [18] Placeholder reference 18.
- [19] Placeholder reference 19.
- [20] Placeholder reference 20.
- [21] Placeholder reference 21.
- [22] Placeholder reference 22.
- [23] Placeholder reference 23.
- [24] Placeholder reference 24.
- [25] Placeholder reference 25.
- [26] Placeholder reference 26.
- [27] Placeholder reference 27.
- [28] Placeholder reference 28.
- [29] Placeholder reference 29.
- [30] Placeholder reference 30.
- [31] Placeholder reference 31.
- [32] Placeholder reference 32.

- [33] Placeholder reference 33.
- [34] Placeholder reference 34.
- [35] Placeholder reference 35.
- [36] Placeholder reference 36.
- [37] Placeholder reference 37.
- [38] Placeholder reference 38.
- [39] Placeholder reference 39.
- [40] Placeholder reference 40.
- [41] Placeholder reference 41.
- [42] Placeholder reference 42.
- [43] Placeholder reference 43.
- [44] Placeholder reference 44.
- [45] Placeholder reference 45.
- [46] Placeholder reference 46.